

SR

SRFashionStyle

Dokumentasi Sistem

Isi: Arsitektur · Kebutuhan · Teknologi · Fitur · Integrasi · Instalasi | Tanggal: 24 Juni 2026

2

APLIKASI

4

ROLE PENGGUNA

3

INTEGRASI

Dokumentasi Sistem — SRFashionStyle

- > **Jenis Dokumen:** Dokumentasi Sistem untuk Laporan Kerja Praktek
- > **Nama Aplikasi:** SRFashionStyle — Platform E-Commerce Fashion Terintegrasi POS
- > **Tanggal:** 24 Juni 2026
- > **Dokumen Terkait:** [Perancangan Basis Data](database-design.md) · [Test Case API](test-case-api.md)

Daftar Isi

1. [Pendahuluan](#1-pendahuluan)
2. [Analisis Kebutuhan Sistem](#2-analisis-kebutuhan-sistem)
3. [Arsitektur Sistem](#3-arsitektur-sistem)
4. [Teknologi yang Digunakan](#4-teknologi-yang-digunakan)
5. [Struktur Proyek](#5-struktur-proyek)
6. [Manajemen Hak Akses (Role)](#6-manajemen-hak-akses-role)
7. [Fitur Sistem per Modul](#7-fitur-sistem-per-modul)
8. [Integrasi Pihak Ketiga](#8-integrasi-pihak-ketiga)
9. [Alur Proses Utama](#9-alur-proses-utama)
10. [Keamanan Aplikasi](#10-keamanan-aplikasi)
11. [Instalasi dan Konfigurasi](#11-instalasi-dan-konfigurasi)

1. Pendahuluan

1.1 Latar Belakang

SRFashionStyle adalah aplikasi web e-commerce untuk penjualan produk fashion yang mengintegrasikan dua kanal penjualan dalam satu sistem: **penjualan online** (toko daring untuk customer) dan **penjualan offline** melalui modul **POS (Point of Sale)** untuk transaksi di toko fisik. Dengan satu basis data terpusat, stok, produk, dan laporan penjualan dari kedua kanal tersinkronisasi secara otomatis.

1.2 Tujuan Sistem

- Menyediakan toko online bagi customer untuk membeli produk fashion dengan pembayaran digital dan pengiriman.
- Menyediakan sistem kasir (POS) bagi staf toko untuk melayani transaksi langsung.
- Memusatkan manajemen produk, varian, inventori, dan laporan dalam satu dashboard admin.
- Mengelola stok secara real-time lintas kanal online dan offline.

1.3 Ruang Lingkup

Sistem terdiri dari dua aplikasi terpisah yang berkomunikasi melalui REST API:

- **Backend** — REST API berbasis Laravel.
- **Frontend** — Single Page Application (SPA) berbasis React.

2. Analisis Kebutuhan Sistem

2.1 Kebutuhan Fungsional

KODE	KEBUTUHAN FUNGSIONAL
KF-01	Sistem dapat melakukan registrasi dan login pengguna dengan verifikasi email OTP
KF-02	Sistem dapat menampilkan katalog produk beserta varian, harga, dan ulasan
KF-03	Customer dapat menambahkan produk ke keranjang dan melakukan checkout
KF-04	Sistem dapat menghitung ongkos kirim melalui integrasi kurir
KF-05	Sistem dapat memproses pembayaran online melalui payment gateway
KF-06	Customer dapat melacak status pesanan dan pengiriman
KF-07	Customer dapat memberikan ulasan setelah pesanan selesai
KF-08	Admin dapat mengelola produk, kategori, varian, dan inventori (CRUD)
KF-09	Admin dapat memilih produk unggulan untuk ditampilkan di homepage
KF-10	Kasir dapat membuka/menutup shift dan mencatat transaksi POS
KF-11	Kasir dapat memproses pembayaran "bayar di toko" untuk order online
KF-12	Admin dapat melihat laporan penjualan dan mengekspornya ke Excel/PDF

2.2 Kebutuhan Non-Fungsional

KODE	KEBUTUHAN NON-FUNGSIONAL
KNF-01	Keamanan — Autentikasi berbasis token (Laravel Sanctum), password ter-hash bcrypt
KNF-02	Pembatasan akses — Otorisasi berbasis role (RBAC) dan policy
KNF-03	Anti-spam — Rate limiting pada endpoint sensitif (login, register, OTP)
KNF-04	Kompatibilitas — Antarmuka responsif untuk desktop dan mobile
KNF-05	Ketersediaan data — Stok tersinkron real-time antara kanal online dan POS

2.3 Kebutuhan Perangkat

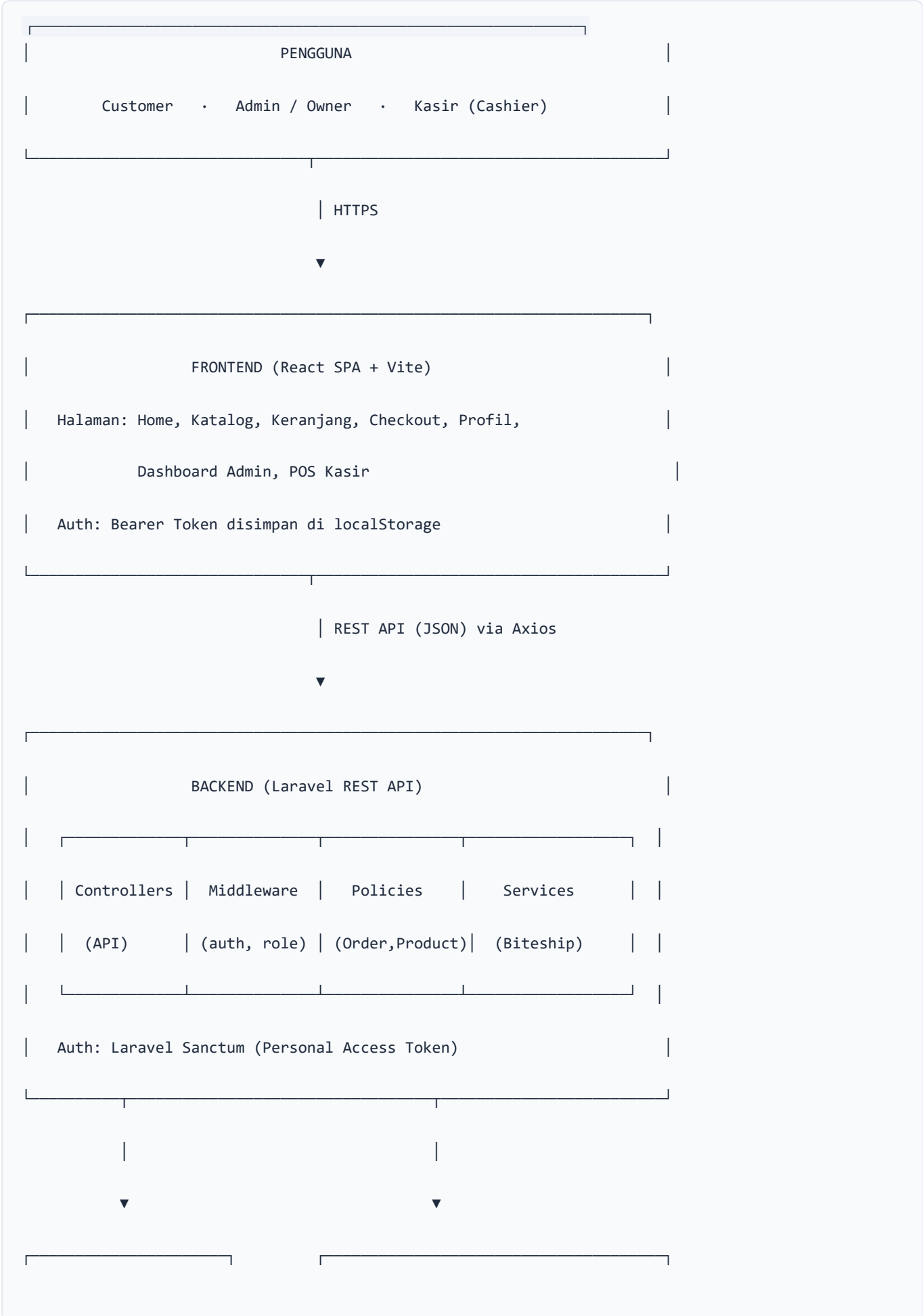
Perangkat Lunak (Pengembangan)

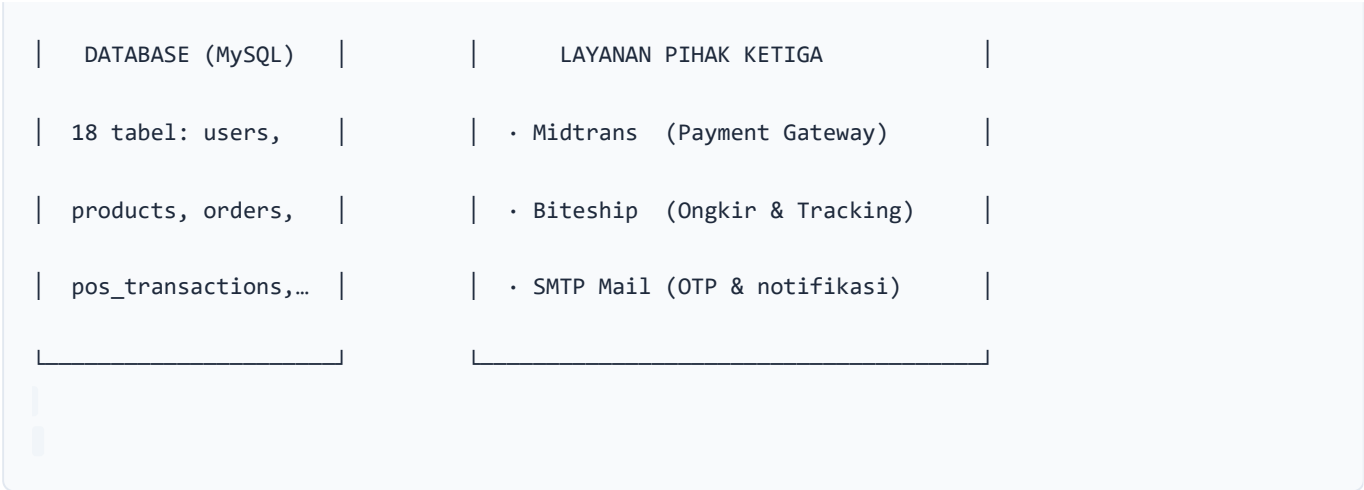
- PHP ≥ 8.1, Composer
- Node.js ≥ 18, npm
- MySQL / MariaDB
- Web server (Apache/Nginx) atau `php artisan serve`

3. Arsitektur Sistem

3.1 Gambaran Umum

Sistem menggunakan arsitektur **client-server terpisah (decoupled)** dengan pola **REST API**. Frontend dan backend berjalan independen dan berkomunikasi melalui HTTP/JSON.





3.2 Pola Komunikasi

- 1. Frontend mengirim request HTTP ke endpoint `/api/*` di backend.
- 2. Backend memvalidasi token (Sanctum) dan role (middleware) pada request terproteksi.
- 3. Controller memproses logika bisnis, berinteraksi dengan database via Eloquent ORM.
- 4. Untuk pembayaran/pengiriman, backend memanggil API pihak ketiga (Midtrans/Biteship).
- 5. Response dikembalikan dalam format JSON ke frontend untuk dirender.

4. Teknologi yang Digunakan

4.1 Backend

TEKNOLOGI	VERSI	FUNGSI
PHP	^8.1	Bahasa pemrograman utama
Laravel Framework	^10.10	Framework backend REST API
Laravel Sanctum	^3.3	Autentikasi API berbasis token
MySQL / MariaDB	—	Sistem manajemen basis data
Midtrans PHP	^2.6	SDK payment gateway
Guzzle HTTP	^7.2	HTTP client (panggil API Biteship)
barryvdh/laravel-dompdf	*	Generate laporan PDF
maatwebsite/excel	*	Generate laporan Excel

4.2 Frontend

TEKNOLOGI	VERSI	FUNGSI
React	^19.2	Library antarmuka pengguna
React Router DOM	^7.15	Routing SPA
Axios	^1.16	HTTP client ke backend API
Vite	^8.0	Build tool & dev server
ESLint	^10.2	Linting kode

5. Struktur Proyek

Proyek menggunakan struktur **monorepo** dengan dua folder utama:

```

SRFashionStyle/
├── backend/                                # Aplikasi Laravel (REST API)
|
|   ├── app/
|   |   ├── Http/
|   |   |   ├── Controllers/Api/          # Controller endpoint API
|   |   |   |   ├── Admin/                # Controller khusus admin/owner
|   |   |   |   ├── Pos/                  # Controller modul kasir
|   |   |   |   └── Store/                 # Controller publik (homepage)
|   |   |   └── Middleware/               # CheckRole, Authenticate, dll.
|   |   ├── Models/                       # Model Eloquent (15 model)
|   |   ├── Policies/                     # OrderPolicy, ProductPolicy
|   |   ├── Services/                     # BiteshipService
|   |   ├── Exports/                      # Kelas export Excel
|   |   ├── Mail/                         # EmailOtpMail
|   |   └── Traits/                       # SlugGenerator
|   ├── database/migrations/              # Skema basis data
|   ├── routes/api.php                    # Definisi seluruh endpoint API
|   └── config/                           # Konfigurasi (sanctum, cors, midtrans)
|
├── frontend/                             # Aplikasi React (SPA)
|   └── src/
|       ├── pages/                        # Halaman per fitur
|       |   ├── admin/                    # Halaman dashboard admin
|       |   ├── customer/                 # Halaman profil customer
|       |   └── pos/                       # Halaman kasir POS

```

```
|   ├── components/           # Layout & komponen reusable
|   ├── lib/axios.js          # Instance Axios + interceptor
|   └── App.jsx                # Definisi routing
|
└── docs/                     # Dokumentasi laporan KP
```

6. Manajemen Hak Akses (Role)

Sistem menerapkan **Role-Based Access Control (RBAC)** melalui middleware `role` dan policy. Terdapat 4 role:

ROLE	DESKRIPSI	AKSES UTAMA
customer	Pelanggan toko online	Katalog, keranjang, checkout, pesanan sendiri, ulasan, profil
cashier	Kasir toko fisik	Modul POS: shift, transaksi, bayar di toko, cari produk
admin	Administrator sistem	Seluruh manajemen produk, order, laporan, kasir, media, + akses POS
owner	Pemilik toko	Sama seperti admin (akses penuh manajemen)

Implementasi: Setiap endpoint terproteksi dibungkus middleware `auth:sanctum` lalu `role:` . Contoh: `role:admin,owner,cashier` untuk manajemen order, `role:customer` untuk pesanan pribadi.

7. Fitur Sistem per Modul

7.1 Modul Autentikasi

- Registrasi akun dengan verifikasi **OTP email** (6 digit, kedaluwarsa 10 menit).
- Login dengan token Sanctum; rate limit 5 percobaan/menit.
- Lupa password & reset password via tautan email.
- Ubah password dan profil pengguna.

7.2 Modul Katalog & Produk (Customer)

- Menampilkan katalog produk aktif dengan varian (warna, ukuran), harga, dan rating.
- Detail produk dengan galeri gambar dan ulasan pembeli.
- Pencarian dan filter produk (kategori, warna, ukuran, harga).
- Section **Best Product** di homepage — produk pilihan admin tampil lebih dulu, sisa slot diisi otomatis dari produk terlaris.

7.3 Modul Keranjang & Checkout (Customer)

- Keranjang belanja berbasis penyimpanan lokal, sinkron antar halaman.
- Checkout dengan dua metode pembayaran: **online (Midtrans)** dan **bayar di toko**.
- Perhitungan ongkos kirim otomatis (Biteship/J&T) atau ambil di toko (pickup).

- Pelacakan status pesanan dan resi pengiriman.
- Konfirmasi pesanan diterima + pemberian ulasan.

7.4 Modul Manajemen (Admin/Owner)

- CRUD **Produk, Kategori, Varian Produk, Inventori**.
- Manajemen **Order** online: ubah status, lihat detail.
- **Best Product** — memilih produk unggulan untuk homepage.
- **Media Promosi** — mengelola banner hero & promo homepage.
- **Kasir (Cashier Staff)** — CRUD data karyawan kasir.
- **Laporan** — laporan penjualan dengan filter tanggal, ekspor Excel & PDF.
- **Dashboard** — ringkasan penjualan online + POS, produk terlaris, shift aktif.

7.5 Modul POS / Kasir (Cashier/Admin/Owner)

- **Shift** — buka shift (modal awal), tutup shift (hitung selisih kas).
- **Transaksi POS** — pencarian produk, keranjang, pembayaran (cash/QRIS/transfer/kartu), cetak struk, refund.
- **Bayar di Toko** — konfirmasi pembayaran untuk order online yang memilih metode bayar di toko.

8. Integrasi Pihak Ketiga

LAYANAN	FUNGSI	IMPLEMENTASI
Midtrans	Payment gateway pembayaran online (Snap)	SDK <code>midtrans/midtrans-php</code> , callback notifikasi pembayaran dengan verifikasi signature SHA512
Biteship	Cek ongkir, buat order kurir (J&T), tracking resi	<code>app/Services/BiteshipService.php</code> via Guzzle HTTP
SMTP Mail	Pengiriman OTP verifikasi email & notifikasi	Mailer Laravel (<code>EmailOtpMail</code>)

9. Alur Proses Utama

9.1 Alur Pembelian Online (Customer)



9.2 Alur Transaksi POS (Kasir)

Kasir → Buka shift (input modal awal)

→ Cari produk → Tambah ke keranjang POS

→ Pilih metode bayar (cash/QRIS/transfer/kartu)

→ Proses transaksi → Stok berkurang → Cetak struk

→ (opsional) Refund transaksi

→ Tutup shift → Sistem hitung total & selisih kas

10. Keamanan Aplikasi

ASPEK	PENERAPAN
Autentikasi	Laravel Sanctum — token dikirim sebagai Authorization: Bearer
Password	Di-hash dengan bcrypt; tidak pernah disimpan plaintext
Verifikasi Email	OTP 6 digit, di-hash di database, kedaluwarsa 10 menit, batas 5 percobaan
Otorisasi	Middleware role + Policy (OrderPolicy, ProductPolicy) cek kepemilikan data
Rate Limiting	Endpoint login/register (5/menit), OTP (3/menit), reset password (5/menit)
CORS	Dibatasi origin frontend; mode token-based (tanpa cookie)
Validasi Input	Setiap request divalidasi di controller sebelum diproses
Webhook	Callback Midtrans diverifikasi via signature SHA512

11. Instalasi dan Konfigurasi

11.1 Backend (Laravel)

```
cd backend
composer install          # Install dependency PHP

cp .env.example .env      # Salin konfigurasi

php artisan key:generate  # Generate app key

# Atur kredensial DB, Midtrans, Biteship, & SMTP di file .env

php artisan migrate       # Buat tabel database

php artisan storage:link   # Symlink storage untuk gambar

php artisan serve         # Jalankan di http://127.0.0.1:8000
```

Variabel `.env` **penting:** `DB_` (database), `MIDTRANS_` (payment), `BITESHIP_API_KEY` (ongkir), `MAIL_*` (email OTP), `FRONTEND_URL`.

11.2 Frontend (React)

```
cd frontend
npm install              # Install dependency

npm run dev              # Jalankan dev server di http://localhost:5173

npm run build            # Build untuk produksi
```

Konfigurasi: `baseURL` API diatur di `src/lib/axios.js` (default `http://127.0.0.1:8000/api`).

11.3 Urutan Menjalankan

1. Jalankan database (MySQL).
2. Jalankan backend (`php artisan serve`).
3. Jalankan frontend (`npm run dev`).
4. Akses aplikasi di `http://localhost:5173`.

> **Catatan:** Dokumen ini melengkapi [Perancangan Basis Data](database-design.md) (ERD, LRS, kamus data) dan [Dokumentasi Pengujian API](test-case-api.md) (167 test case). Ketiganya membentuk dokumentasi teknis lengkap untuk laporan Kerja Praktek.